

DEMO disclosure: This manuscript uses original synthetic editorial-evaluation materials. Its numerical results are authored fixtures, not real experimental measurements.

Sparse Raster Transfer with a Coherent Publication Contract

Abstract

Sparse raster transfer saves communication only if the client can reuse unchanged tiles without exposing a mixture of producer revisions. We describe a staged-patch protocol for a renderer whose immutable tile versions and atomic root publication already exist. The protocol makes three decisions explicit: which acknowledged base may be reused, which target values a coalesced update must carry, and when delivery permits publication. Full snapshots provide an equally coherent comparison; immediate tile installation diagnoses the failure of revision tags alone. In authored constrained-link fixtures, sparse patches reduce median wire volume from 1026 to 68 KiB and packet-completion delay from 87 to 17 ms while preserving coherence at every prescribed reader interleaving. The advantage reverses for dense changes and for sparse changes on a fast local link. The demonstration therefore supports a conditional integration argument, with explicit fallback and retention costs, rather than empirical performance or first-of-kind claims.

1. Introduction: transfer savings under a view constraint

An environmental raster dashboard displays the latest committed revision available locally while newer values arrive. Incremental delivery reduces redundant traffic when most tiles remain unchanged. It also creates a consistency problem: installing each received tile immediately can reveal combinations that the producer never published.

The existing application already gives local readers a stable view. Each frame captures a published root containing pointers to immutable tile versions. Atomic publication, root copying, reference counting, sparse invalidation, and tile checksums predate this work. Computation of raster values is outside the study. The transfer problem is to preserve this established view contract as clients pause, disconnect, or fall behind.

Our design stages a patch against a named base and publishes only a complete, validated target root. Its substantive implementation work connects acknowledgement, retained history, target-value coalescing, and failure handling to that publication boundary. Comparing against staged full snapshots exposes the actual tradeoff: both preserve coherent views, but sparse updates exchange reduced payload for additional preparation and revision-state management. The supplied synthetic fixtures make this tradeoff inspectable without establishing deployment suitability or research novelty.

2. Method: reconstruct a target before exposing it

2.1. Reuse a named base

A raster comprises 64 tiles, each containing 4096 four-byte cells. Its full payload is 1,048,576 bytes. A root holds the 64 tile-version pointers. Target coherence means matching the producer's target root; an unchanged tile created at revision 8 can legitimately appear in revision 11.

Three implementations differ in what they transfer and expose. Full snapshots (FS) stage the complete raster, validate all packets, then switch the visible root. Immediate tiles (IT) transfer changed tiles with target revision tags and install each pointer on arrival. Staged patches (SP) use a manifest naming the base, target, and changed-tile set. The client pins its visible base, copies its pointers into a private pending root, and replaces declared changed tiles. FS and SP both use the existing root-publication operation.

For a two-tile illustration, revision 10 contains (a0, b0), and revision 11 requires (a1, b1). After only a1 arrives, IT exposes (a1, b0), matching neither root. SP keeps (a0, b0) visible until both replacements validate, then publishes (a1, b1). An unchanged third tile can remain shared throughout. This is a semantic example, not another timing result. Pending roots remain invisible; readers need not wait for pending packets. No requirement makes remote clients display the same revision at the same wall-clock instant.

2.2. Coalesce target values, then acknowledge publication

The client acknowledges only the root it has published. The server retains manifests for at most eight completed revisions and derives a patch from that acknowledged base when available. Combining manifest steps requires the union of touched coordinates, with each transmitted value taken from the target root. An earlier intermediate value is insufficient. A tile that changes and later returns to its original bytes may remain in the union: coalescing is correct without being a minimal-difference compressor.

A commit checks the pending patch against its pinned base before publication. An unavailable, wrong, or unverifiable base triggers FS fallback. Reordered packets may enter staging under their named target; matching duplicates are idempotent, while conflicting duplicates reject the patch. Size, checksum, and tile-ID validation guard assembly. Incomplete or invalid pending patches are discarded, leaving the last coherent view visible. Disconnect discards uncommitted staging. Even a valid late packet must not update a newer visible root.

These rules make transport progress usable for subsequent updates. Acknowledging receipt of the first packet can advance the server's assumed base before the client has published it. The protocol manages volatile display state; it supplies neither durable transactions nor process-crash recovery.

2.3. Pay for staging and retention

A pending update adds a 512-byte pointer table and 4096 bytes of descriptors and reference counters. SP retains old tiles plus replacements until readers release the old root; FS retains an old frame and a new complete frame. Dense SP changes can require a full frame of pending payload. Old-reader pinning has no proven lifetime bound. Manifest validation and root assembly also consume CPU time, so smaller transfers need not complete sooner.

3. Evaluation: benefits and reversals in authored fixtures

All table entries are synthetic. Each row covers the same 120 target roots; within each workload/link pair, methods use identical lossless values and packet schedules. Compression is disabled. Queue capacity and retry deadline remain 32 packets and 20 ms. The earlier 64-packet exploratory run is excluded. Serialization and transport differ only where payload selection and manifest handling require it.

Completion delay runs from producer target publication until all packets needed for that target are applied or staged. It is neither first-byte latency nor a measurement of frame rendering. The coherence count records runs with no non-target mixed root at any prescribed reader interleaving from request through completion. Retaining the prior coherent root while staging is permitted. The interleavings form a finite fixture, not an exhaustive scheduling proof.

Changed tiles / link	Method	Coherent / 120	Median / p95 completion (ms)	Median wire (KiB)	Peak payload (MiB)	Median preparation (ms)
Sparse 4 / constrained	FS	120	87 / 106	1026	2.00	0.18
Sparse 4 / constrained	IT	71	13 / 26	66	1.06	0.05
Sparse 4 / constrained	SP	120	17 / 31	68	1.06	0.32
Moving 24 / constrained	FS	120	89 / 110	1026	2.00	0.18

Changed tiles / link	Method	Coherent / 120	Median / p95 completion (ms)	Median wire (KiB)	Peak payload (MiB)	Median preparation (ms)
Moving 24 / constrained	IT	40	47 / 69	386	1.38	0.07
Moving 24 / constrained	SP	120	51 / 74	388	1.38	0.37
Dense 60 / constrained	FS	120	91 / 111	1026	2.00	0.18
Dense 60 / constrained	IT	31	83 / 104	962	1.94	0.09
Dense 60 / constrained	SP	120	97 / 120	964	1.94	0.44
Sparse 4 / fast local	FS	120	2.2 / 3.1	1026	2.00	0.18
Sparse 4 / fast local	IT	71	1.3 / 2.2	66	1.06	0.05
Sparse 4 / fast local	SP	120	2.6 / 3.9	68	1.06	0.32

KiB and MiB are binary. Wire volume includes protocol framing. Peak retained payload excludes pointer tables and descriptors and assumes one reader finishes within one display cycle.

3.1. Compare designs that preserve the same contract

FS and SP preserve coherence in all 120 runs of each case. IT does so in only 71, 40, and 31 constrained-link runs for sparse, moving, and dense changes; its fast-local sparse count is also 71. Its lower completion delays diagnose what immediate installation buys while permitting mixed views, and cannot establish a coherence-preserving performance baseline.

Relative to FS, SP cuts sparse constrained-link wire volume by 93.4% and median completion delay by 80.5%. For 24 moving tiles, the corresponding values are 388 versus 1026 KiB and 51 versus 89 ms. Both cases also improve p95. Retained payload rises with patch density: SP needs 1.06, 1.38, and 1.94 MiB across the constrained cases, approaching FS's 2.00 MiB.

The counterexamples are equally consequential. With 60 changed tiles, SP reduces wire volume only to 964 KiB and worsens median/p95 completion to 97/120 ms versus FS's 91/111 ms. Even the sparse fast-local case reverses the latency result: 2.6/3.9 ms for SP versus 2.2/3.1 ms for FS. SP preparation exceeds FS preparation in every row. Additional assembly work is consistent with this cost, but the fixtures do not isolate causes or attribute every latency difference solely to bytes.

3.2. Check revision-state rules separately

In 28 hand-authored functional traces, SP matches the target-root oracle throughout the supplied suite. Coverage includes first connection without a base, empty patches, delayed final tiles, reordering, duplicates and conflicts, corruption, disconnect during staging, late reconnect packets, base eviction, and two pinned readers. Eight eviction/reconnect traces take FS fallback. No recovery latency summary exists, and those traces cannot establish sparse-path byte savings during fallback.

Two development variants fail: omitting base checks exposes mismatched roots in two of six reconnect traces; acknowledging the first packet instead of publication fails after coalescing in one of six. These failures motivate the corresponding rules but are not timing ablations.

4. Discussion and conclusion

The useful design boundary is where reduced transfer outweighs staging work while the renderer retains a coherent root. The fixtures illustrate both sides without defining a general switching threshold. Only sparse changes have a fast-local case; each case uses one fixed schedule, without repetitions, confidence intervals, stochastic loss modeling, or CPU microarchitecture analysis. Payload figures cannot bound memory for stalled

readers. Allocation-failure injection and process-crash tests are absent. Security, authentication, and Byzantine behavior are outside scope.

The demonstration ties safe base reuse, target-value coalescing, and publication-based acknowledgement to an existing immutable-root renderer. Full snapshots remain a coherent alternative and the required recovery path when the base cannot be trusted. With no literature survey or real benchmark, the supported result is this bounded design and comparison, not universal novelty, real-world speed, or deployment readiness.